

Optional Features Implementation

This document describes the optional stretch goals implemented in the Toy Blockchain project.

Currently implemented optional features:

1. Digital Signatures
2. Merkle Root

These features improve transaction authenticity and block integrity.

1. Digital Signatures

Overview

Digital signatures provide a way to verify that a transaction was created by the actual owner of an account.

Without digital signatures, anyone could create a transaction pretending to be another user:

```
Alice -> Bob : 100
```

The blockchain would have no way to verify whether Alice actually approved the transaction.

With digital signatures, every transaction contains cryptographic proof created using the sender's private key.

Implementation Details

Key Generation

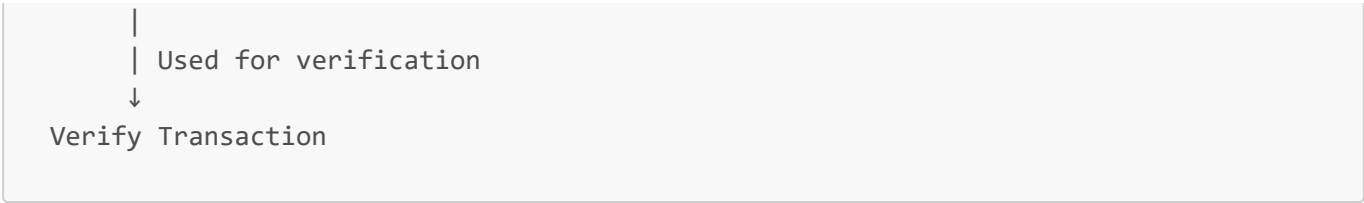
The project uses:

- ECDSA (Elliptic Curve Digital Signature Algorithm)
- P-256 elliptic curve

A key pair is generated:

```
Private Key
  |
  | Used for signing
  ↓
Transaction Signature

Public Key
```



The private key remains with the owner, while the public key is attached to the transaction.

Transaction Structure

Before digital signatures:

```
type Transaction struct {
    Sender      string
    Recipient   string
    Amount      float64
}
```

After adding digital signatures:

```
type Transaction struct {
    Sender      string
    Recipient   string
    Amount      float64

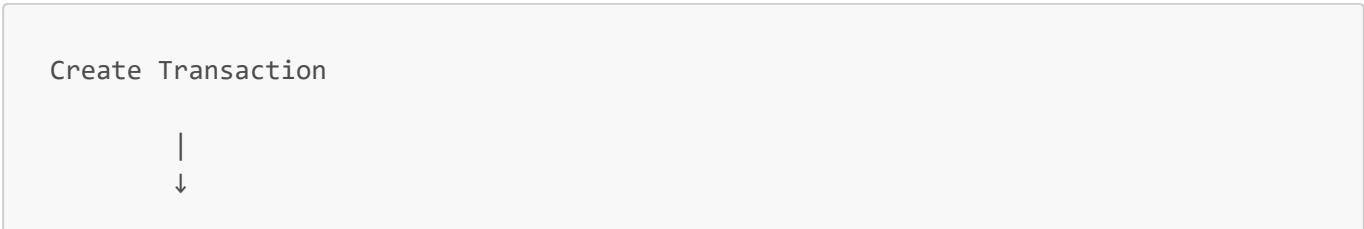
    PublicKey   string
    Signature    string
}
```

Additional fields:

Field	Purpose
PublicKey	Used to verify the sender's identity
Signature	Cryptographic proof of transaction approval

Transaction Signing Flow

The transaction process now works as follows:



Generate Signature using Private Key



Attach Public Key and Signature



Verify Signature



Add Transaction to Pending Pool



Mine Block

Signature Verification

Before adding a transaction into the pending transaction pool, the blockchain performs signature verification.

The validation process:

Transaction Received



Calculate Transaction Hash



Verify Signature using Public Key



Valid?

|
+----- No ----> Reject Transaction

```
|  
+---- Yes --> Continue Validation
```

Invalid transactions are rejected with:

```
invalid transaction signature
```

How To Test Digital Signatures

1. Start Blockchain

Run:

```
go run ./cmd/toy-blockchain
```

2. Create Initial Balance

Select:

```
6. Faucet (Create Coinbase Transaction)
```

Example:

```
Recipient: Alice  
Amount: 100
```

3. Mine Coinbase Transaction

Select:

```
4. Mine Transactions
```

4. Create Signed Transaction

Select:

2. Add Transaction

Example:

```
Sender: Alice  
Recipient: Bob  
Amount: 20
```

Expected output:

```
Transaction added
```

5. Mine Transaction

Select:

```
4. Mine Transactions
```

Expected:

```
Block mined successfully
```

6. Validate Blockchain

Select:

```
5. Validate Blockchain
```

Expected:

```
Blockchain is valid
```

Security Benefit

If someone modifies transaction data after signing:

Example:

Original:

```
Alice -> Bob : 20
```

Changed:

```
Alice -> Bob : 200
```

The signature no longer matches the transaction data.

Verification fails because:

```
Modified transaction hash != Original signed hash
```

Therefore the transaction is rejected.

2. Merkle Root

Overview

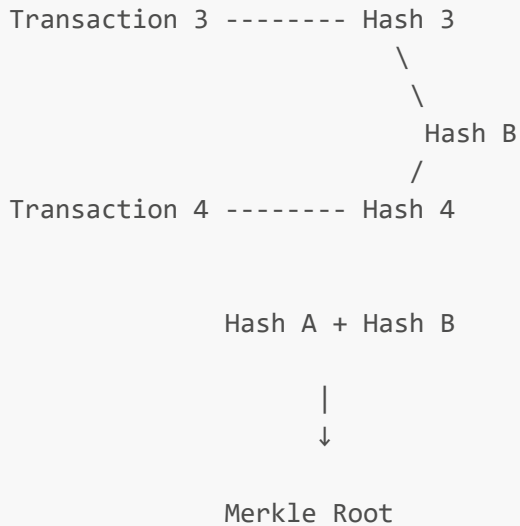
A Merkle Root summarizes all transactions inside a block using a tree-based hashing structure.

Instead of directly hashing every transaction inside the block, the blockchain calculates a single hash value representing all transactions.

Merkle Tree Structure

Example with four transactions:

```
Transaction 1 ----- Hash 1
                        \
                         \
                        Hash A
                         /
Transaction 2 ----- Hash 2
```



The Merkle Root changes whenever any transaction changes.

Implementation Details

Merkle Package

A new package was added:

```
internal/blockchain/merkle
```

The main function:

```
CalculateMerkleRoot(transactions []transaction.Transaction) string
```

This function:

1. Hashes each transaction
2. Combines transaction hashes in pairs
3. Continues hashing until one final hash remains
4. Returns the Merkle Root

Block Structure Update

Before:

```
type Block struct {  
    Index
```

```
    Timestamp
    Transactions
    PreviousHash
    Nonce
    Hash
}
```

After:

```
type Block struct {
    Index
    Timestamp
    Transactions
    PreviousHash
    Nonce
    MerkleRoot
    Hash
}
```

Each block now stores the Merkle Root of its transactions.

Mining Flow With Merkle Root

The mining process now:

```
graph TD
    A[Pending Transactions] --> B[Calculate Merkle Root]
    B --> C[Create Candidate Block]
    C --> D[Calculate Block Hash]
    D --> E[Proof-of-Work Mining]
```



Add Block To Chain

Hashing Changes

Before:

Block Hash Input:

Index
Timestamp
Transactions
Previous Hash
Nonce

After:

Block Hash Input:

Index
Timestamp
Merkle Root
Previous Hash
Nonce

The block hash now depends on the Merkle Root instead of directly depending on the complete transaction list.

Validation Changes

Blockchain validation now checks:

1. Block hash integrity
2. Previous block linkage
3. Proof-of-work difficulty
4. Block index ordering
5. Timestamp ordering
6. Merkle Root integrity

Validation recalculates:

```
Transactions
  |
  ↓
Calculate Merkle Root
  |
  ↓
Compare with stored Merkle Root
```

If they differ:

```
block X merkle root mismatch
```

The blockchain is considered invalid.

How To Test Merkle Root

1. Start using a new blockchain file

Run:

```
go run ./cmd/toy-blockchain -data merkle-test.json
```

2. Create Coinbase Transaction

Select:

```
6. Faucet
```

Example:

```
Recipient: Alice
Amount: 100
```

3. Mine Block

Select:

4. Mine Transactions

4. Add Transaction

Select:

2. Add Transaction

Example:

```
Sender: Alice
Recipient: Bob
Amount: 20
```

5. Mine Again

Select:

4. Mine Transactions

6. View Blockchain

Select:

1. View Blockchain

Expected output:

```
Block #1

Previous Hash : <hash>
Merkle Root   : <hash>
Hash          : <hash>
Nonce         : <value>
```

7. Validate Blockchain

Select:

```
5. Validate Blockchain
```

Expected:

```
Blockchain is valid
```

Summary

Implemented optional features:

Feature	Status
Digital Signatures	Completed
ECDSA Key Generation	Completed
Transaction Signing	Completed
Signature Verification	Completed
Merkle Root Calculation	Completed
Merkle Root Storage in Block	Completed
Merkle Root Validation	Completed

These features improve the blockchain by adding:

- Transaction authenticity
- Protection against unauthorized transactions
- Efficient transaction integrity verification
- Stronger block validation

3. Concurrent Mining

Overview

Concurrent mining improves the Proof-of-Work process by utilizing multiple CPU cores. Instead of searching for a valid nonce using a single execution thread, the mining workload is divided among several Go goroutines running in parallel.

This allows multiple nonces to be tested simultaneously, reducing the expected mining time while demonstrating Go's concurrency capabilities.

Implementation Details

Worker Creation

The miner automatically determines the number of available CPU cores using:

```
workers := runtime.NumCPU()
```

One goroutine is created for each available CPU core.

Nonce Distribution

Instead of every worker checking the same nonces, each worker searches a different section of the nonce space.

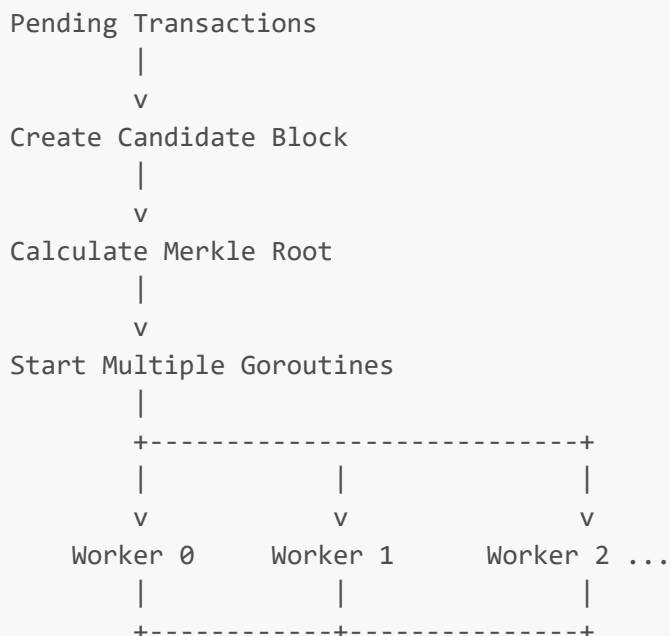
Example with four workers:

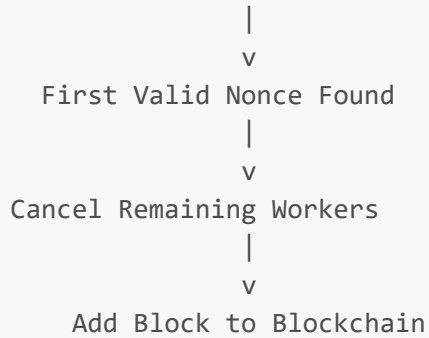
```
Worker 0: 0, 4, 8, 12, 16...
Worker 1: 1, 5, 9, 13, 17...
Worker 2: 2, 6, 10, 14, 18...
Worker 3: 3, 7, 11, 15, 19...
```

This prevents duplicate work and allows the entire nonce space to be searched more efficiently.

Mining Flow

The mining process now follows these steps:





Goroutine Coordination

The implementation uses several Go concurrency primitives:

Component	Purpose
<code>goroutines</code>	Perform Proof-of-Work in parallel
<code>context.Context</code>	Stops all workers once a solution is found
<code>sync.WaitGroup</code>	Waits for every worker to exit cleanly
<code>channel</code>	Returns the first successfully mined block

Worker Termination

When one worker finds a valid hash:

1. The mined block is sent through a channel.
2. A cancellation signal is broadcast using `context.CancelFunc`.
3. Remaining workers stop immediately.
4. The main goroutine waits for all workers to exit.
5. The mined block is returned.

This ensures only one valid block is accepted while avoiding unnecessary computation.

Advantages

Compared to single-threaded mining, concurrent mining provides:

- Better CPU utilization
 - Parallel nonce searching
 - Faster Proof-of-Work on multi-core processors
 - Clean worker shutdown after a solution is found
-

How to Test

1. Start the blockchain

```
go run ./cmd/toy-blockchain
```

2. Create a transaction

Choose:

```
6. Faucet (Create Coinbase Transaction)
```

Example:

```
Recipient: Alice  
Amount: 100
```

Mine the transaction.

3. Create another transaction

Choose:

```
2. Add Transaction
```

Example:

```
Sender: Alice  
Recipient: Bob  
Amount: 20
```

4. Mine the block

Choose:

```
4. Mine Transactions
```

The mining operation now performs Proof-of-Work using multiple goroutines running concurrently.

5. Validate the blockchain

Choose:

```
5. Validate Blockchain
```

Expected output:

```
Blockchain is valid
```

Summary

Concurrent mining introduces parallel Proof-of-Work without changing the blockchain's external behavior.

The blockchain still produces exactly one valid block for a set of transactions, but the nonce search is distributed across multiple goroutines, improving performance and demonstrating Go's native concurrency features.